
HPDP PROJECT 1 SECTION 02

# OPTIMIZING HIGH-PERFORMANCE DATA PROCESSING FOR LARGE-SCALE WEB CRAWLERS

GROUP BigPotato

# INTRODUCTION

The Malaysian job market generates a large volume of online recruitment data through platforms such as Jobstreet. However, collecting this information manually is inefficient and time-consuming. This project develops a High-Performance Computing (HPC)-based web scraper that automates large-scale data collection using parallel processing techniques. The optimized system enables faster and more efficient extraction of over 100,000 job records, providing structured data for job market analysis and performance evaluation.

**Extract 100,000+ job records from Jobstreet.**

**Build an automated pipeline for data cleaning and preprocessing.**

**Optimize performance using:**

- **Multithreading**
- **Multiprocessing**
- **Asynchronous programming**

**Compare execution performance before and after optimization.**

# DATA COLLECTION

# COLLECTION PROCESS

## Data Source

target

- 100000+ records
- Job Title
- Company
- Location
- Salary
- Classification

## Crawling Engine

**Runs 8 concurrent workers**

**Loads JobStreet Website**

**Extracts job information from the HTML**

**Pagination Loop – Repeat process for every page and job classification**

## Output

- 105094 rows records
- JSONL file

# RAW DATA

{"job_title": "AI Engineer / AI Specialist (1-Year Renewable Contract @MidValley Southkey)", "company": "HopeRun Software Singapore Pte Ltd", "location": "Johor Bahru", "classification": "(Information & Communication Technology)", "salary": "RM 6,800 – RM 10,000 per month"}

# DATA PROCESSING

# DATA CLEANING

- Removed duplicates: from 105,094 to 26,088 rows
- Dropped rows with missing required fields
- Filled missing salary as “Not Disclosed” instead of removing them
- After missing handling handling data cleaning, remaining 26,074 rows

# DATA TRANSFORMATION

- Cleaned whitespace
- Standardized classification format
- Standardized locations like KL to be Kuala Lumpur
- Removed non-ASCII job titles like chinese words
- Extracted new columns, salary_min and salary_max

## RAW DATA

105, 094 rows
5 columns

→

## CLEANED DATA

25, 209 rows
7 columns

# CLEANED DATA

| No. | Attribute | Description | Example |
|---|---|---|---|
| 1 | Job Title | Name of the job position | Accountant |
| 2 | Company | Hiring company name | AECOM |
| 3 | Location | Job location | Johor Bahru |
| 4 | Classification | Job category or industry | Engineering |
| 5 | Salary | Original salary text | RM 2,400–RM 3,400 per month |
| 6 | Salary Min | Minimum salary value | 2400 |
| 7 | Salary Max | Maximum salary value | 10500 |

# OPTIMIZATION TECHNIQUES

# OVERVIEW

| | |
|---|---|
| **Pandas** | Memory optimization, vectorized operations, efficient grouping |
| **Polars** | Lazy execution, query optimization, parallel processing |
| **DuckDB** | Direct CSV querying, SQL aggregation, analytical processing |

# FRAMEWORK EXPLANATION

## PANDAS OPTIMIZED PIPELINE

- Loads only required columns using usecols
- Reduces memory using dtype and category
- Uses vectorized string processing
- Applies optimized groupby(observed=True)

## POLARS LAZY EXECUTION

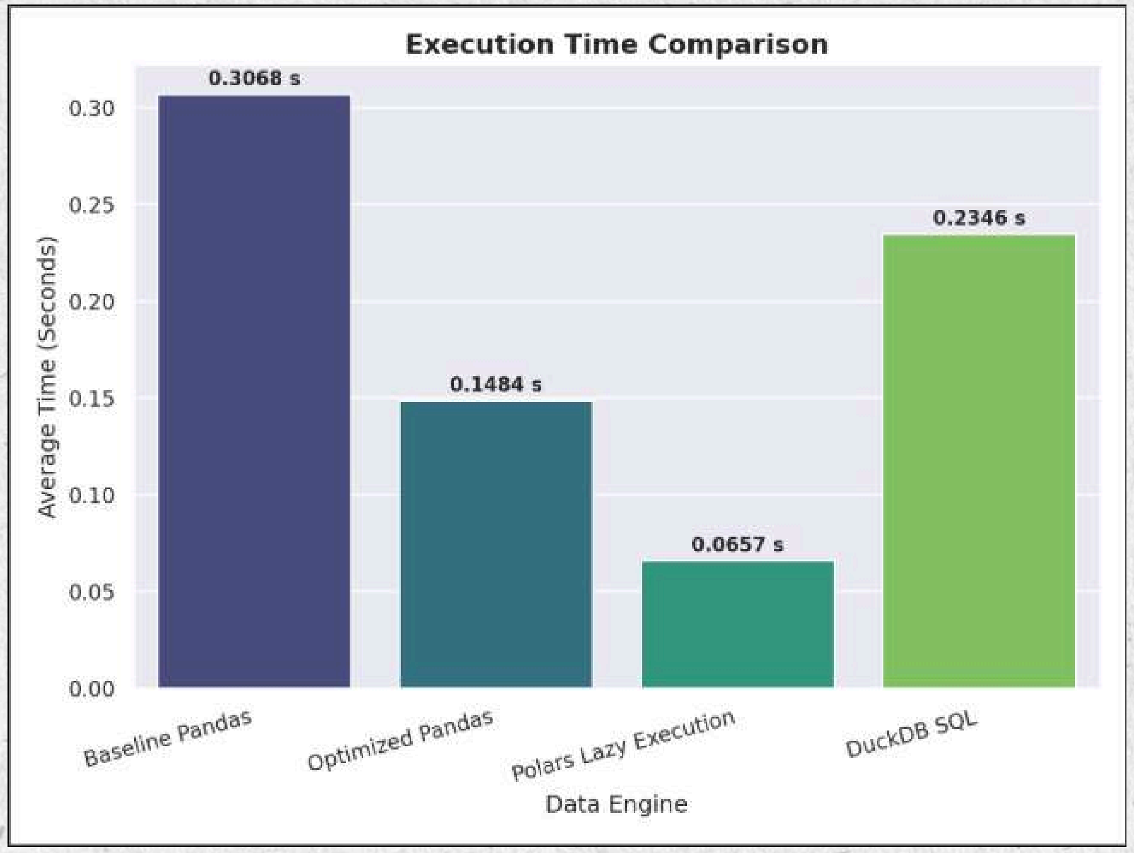
- Uses scan_csv() for lazy loading
- Executes only when .collect() is called
- Supports parallel aggregation
- Suitable for fast DataFrame processing

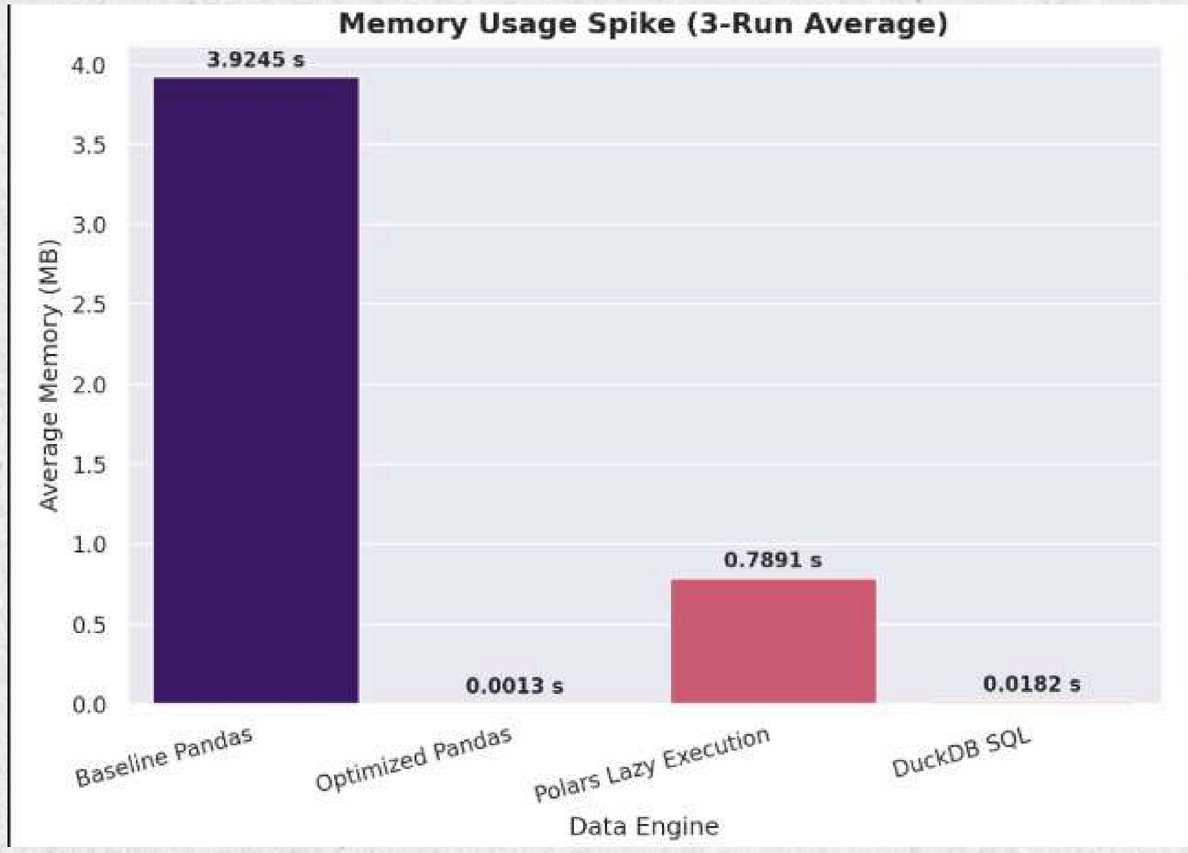
## DUCKDB SQL PROCESSING

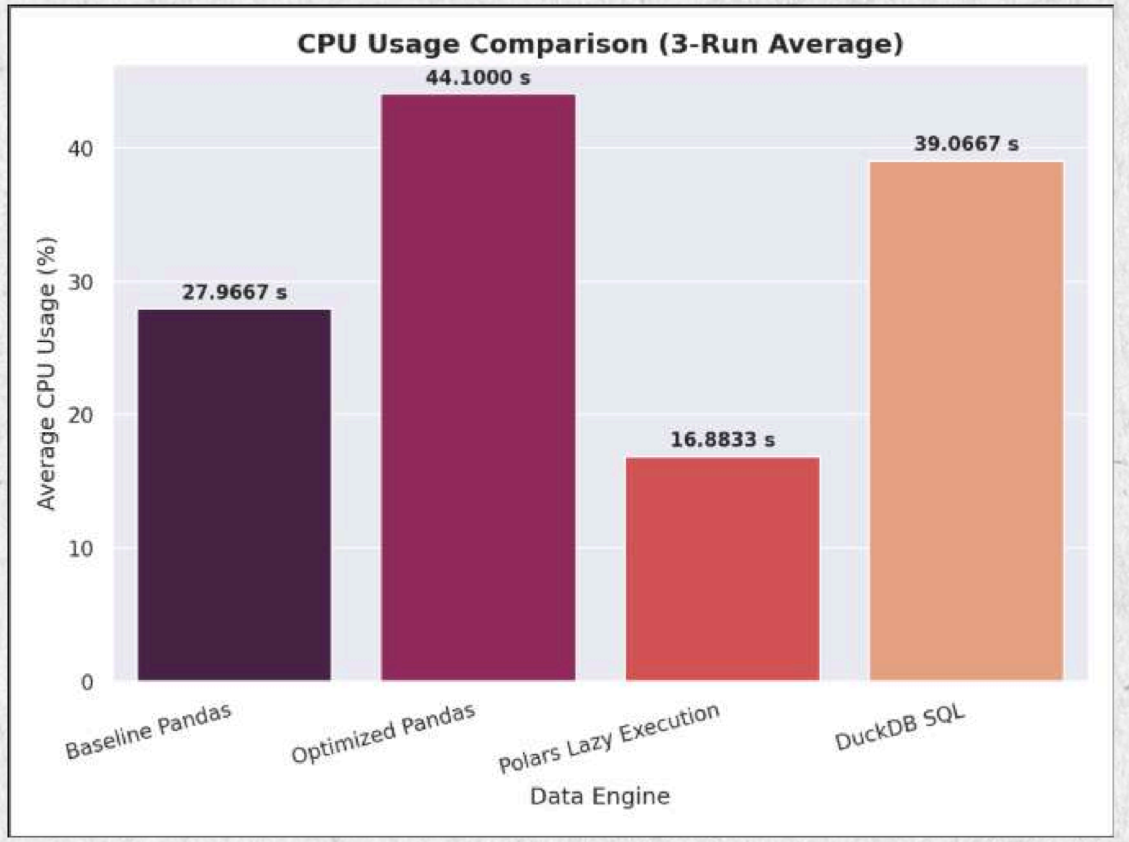
- Queries CSV directly using read_csv_auto()
- Uses SQL filtering and aggregation
- Applies GROUP BY, COUNT(), and COUNT(DISTINCT)
- Suitable for analytical queries without fully loading data into memory

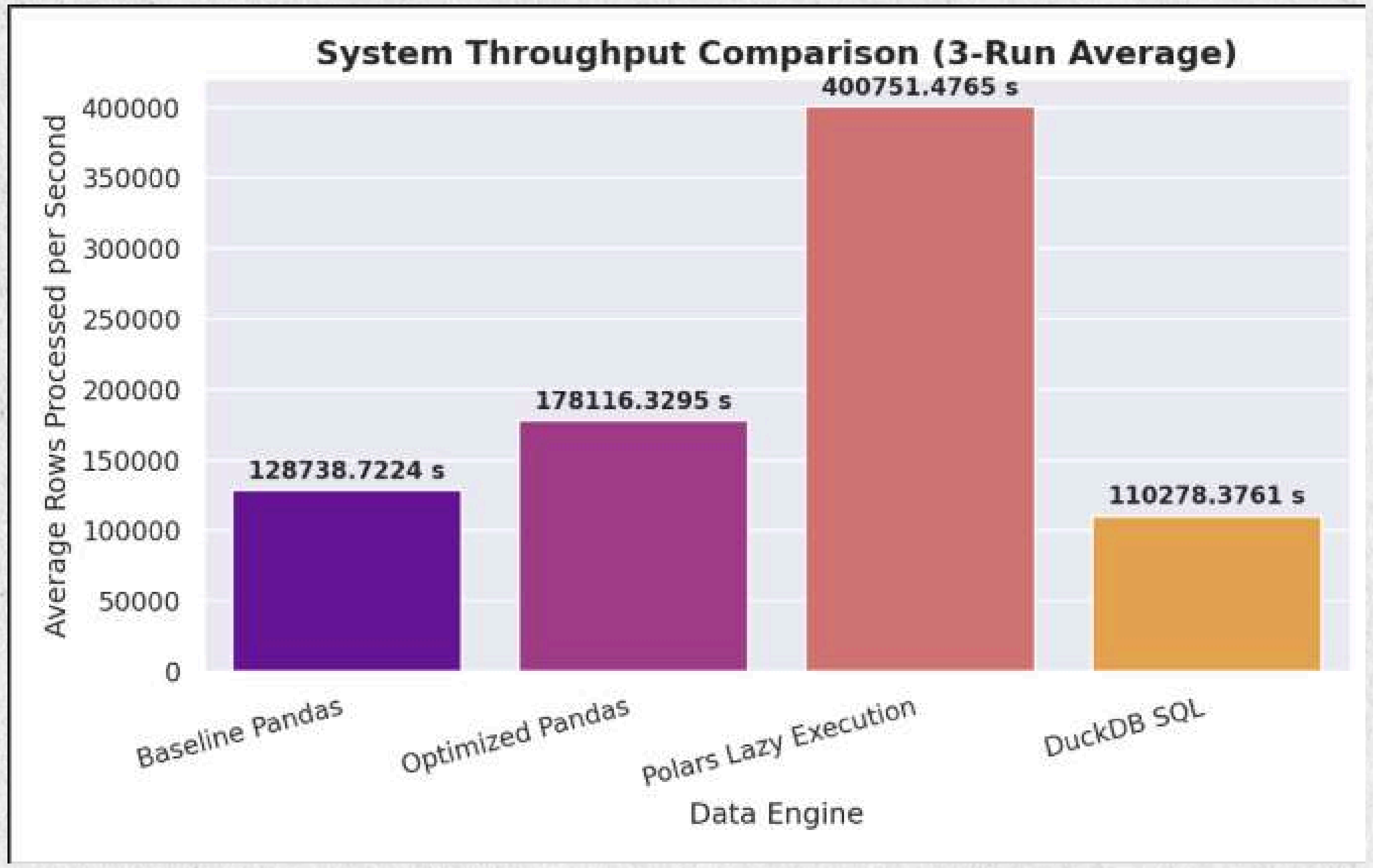
# PERFORMANCE EVALUATION

# VISUALIZATION



# COMPARISON

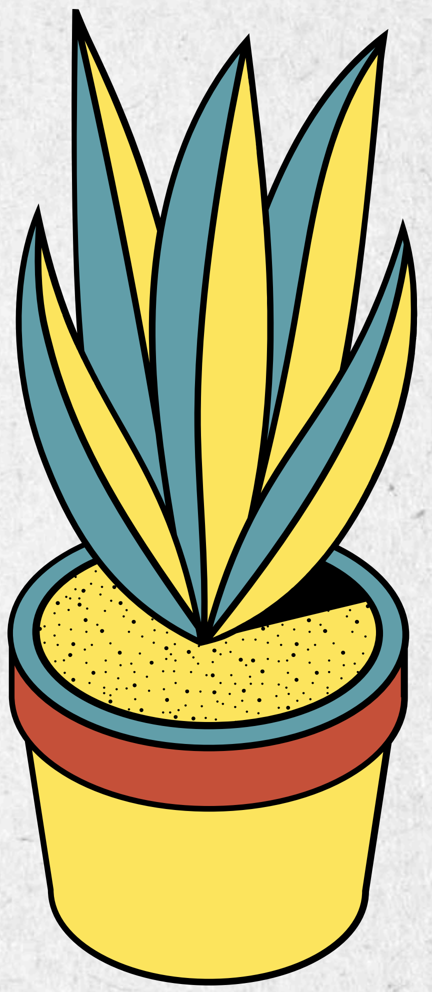
| PROCESSING FRAMEWORK | AVG EXECUTION (S) | AVG PEAK MEMORY (MB) | AVG CPU USAGE (%) | THROUGHPUT |
|---|---|---|---|---|
| POLARS LAZY EXECUTION | **0.07** | 0.79 | **16.88** | **400751.48** |
| OPTIMIZED PANDAS | 0.15 | **0.001** | 44.10 | 178116.33 |
| BASELINE PANDAS | 0.31 | 3.92 | 27.99 | 128736.72 |
| DUCKDB SQL | 0.23 | 0.02 | 39.07 | 110278.38 |

# SUMMARY

- Built a complete JobStreet data pipeline
- Cleaned 105,094 records into 25,209 rows
- Produced a cleaned dataset with 25,209 rows and 7 columns.
- Compared four processing approaches: Baseline Pandas, Optimized Pandas, Polars Lazy Execution and DuckDB SQL.
- Polars achieved the best overall performance
- Optimization improved speed and efficiency

# FUTURE WORKS

- Benchmark using 1M+ records
- Use a more reliable cloud server
- Add complex joins and transformations
- Replace CSV with Parquet
- Store data in PostgreSQL or MongoDB
- Automate workflows using Airflow
- Add incremental crawling



# THANK YOU